

Migrating Legacy SCM and Defects to GitHub

Modernizing development workflows with advanced tools

Why This Migration Matters

Legacy Limitations

Old SCM platforms limit development with centralized check-ins and costly branching, hindering collaboration and integration.

Value of Institutional Memory

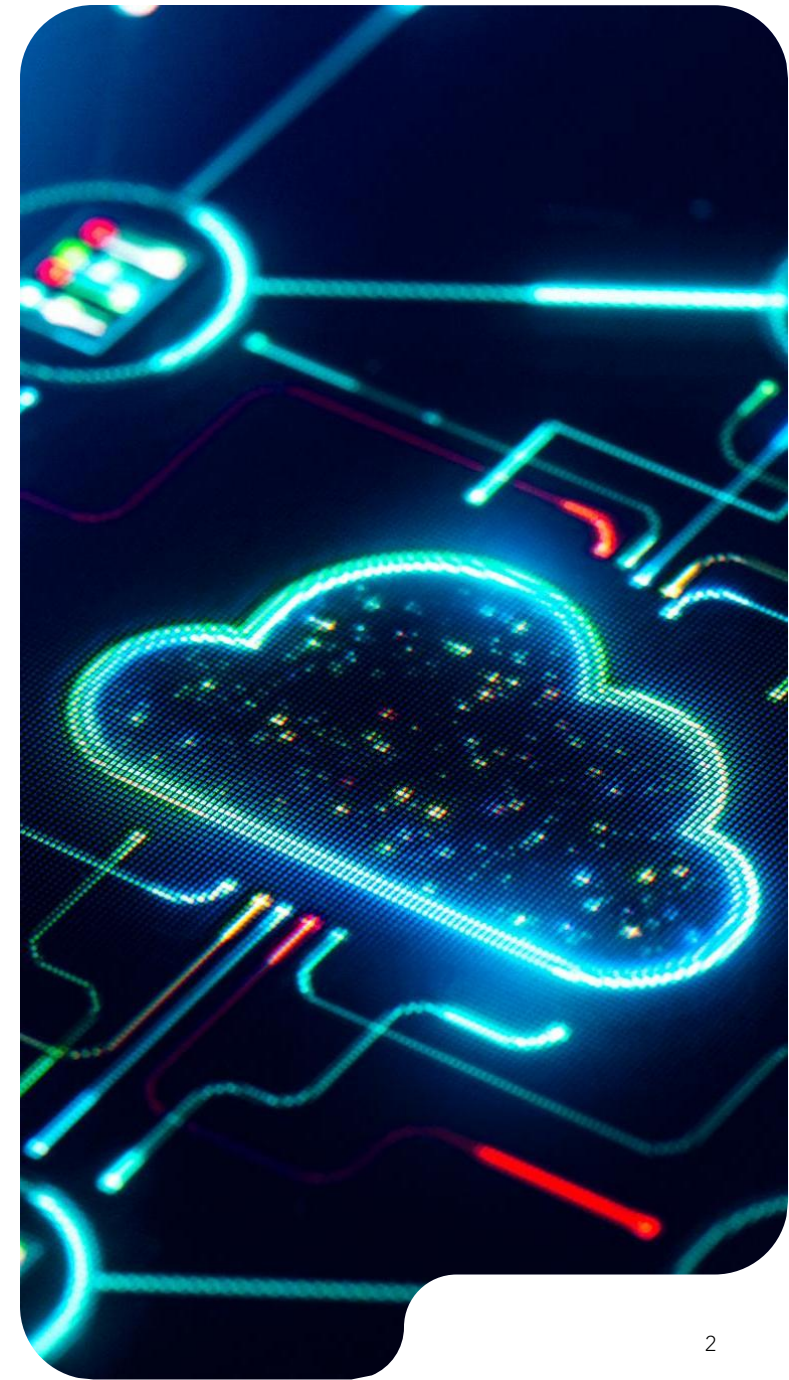
Preserving source history and defect data protects engineering insights vital for accountability and regulatory compliance.

Benefits of Modern Migration

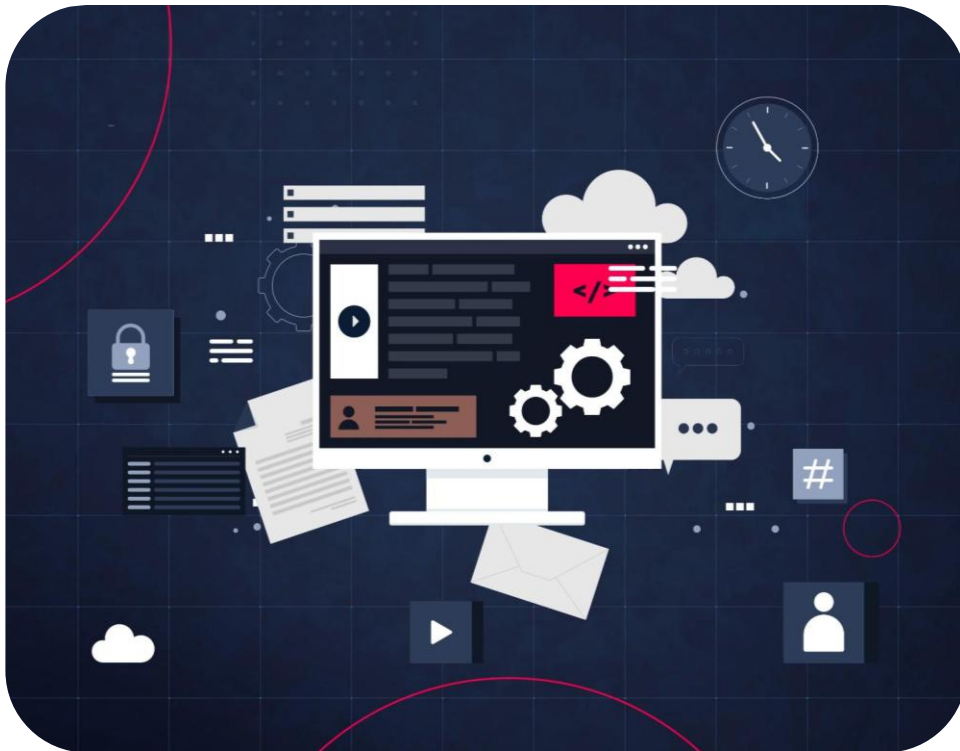
Migrating to GitHub enables distributed workflows, automation, and better developer experience if critical data is maintained.

Balanced Migration Approach

Successful migration balances modernization with preserving engineering continuity to ensure stakeholder alignment and trust.



Migration Scope: Beyond Source Code



Comprehensive Data Migration

Migration includes source code files, versions, branches, and releases mapped to Git entities like commits and tags.

Defect Data Integration

Defect data with identifiers, statuses, priorities, and comments must be migrated to preserve defect lifecycle context.

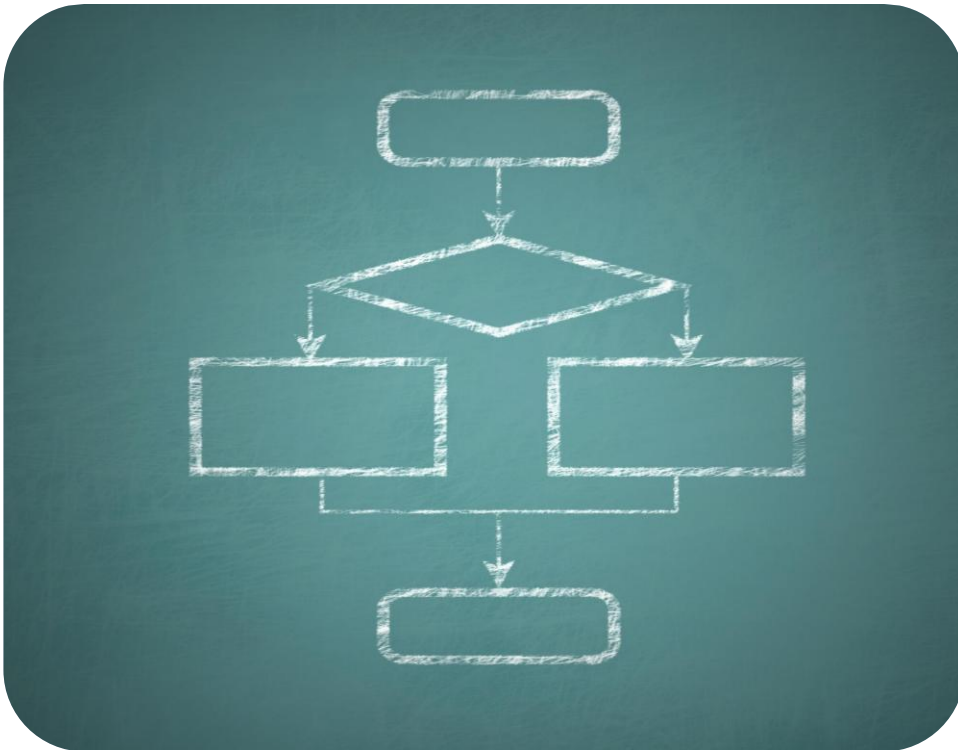
Traceability Links

Linking defects to changesets and releases maintains traceability essential for audits and impact analysis.

Collaboration History

Preserving timestamps and authorship data supports forensic analysis and audit requirements post-migration.

Target Migration Architecture



Extraction Phase

Data is extracted from CMVC, including versions, history, defects, and relationships for migration.

Transformation Phase

Extracted data is normalized and mapped to structures compatible with GitHub.

Load Phase

Transformed data is converted into Git commits and defects become GitHub issues via APIs.

Validation Phase

Validation ensures data integrity by comparing counts, sampling metadata, and verifying traceability.

Preserving Source History as Git Commits

Accurate Authorship and Timing

Each CMVC change is converted into a Git commit preserving original authorship and timestamp for accountability.

Preserving Branching and Tags

Branch structures and release levels are recreated as Git branches and tags for historical reference.

Embedding Defect References

Commit messages embed original defect IDs linking code changes directly to issues.



Migrating Defects to GitHub Issues

Defect Context Preservation

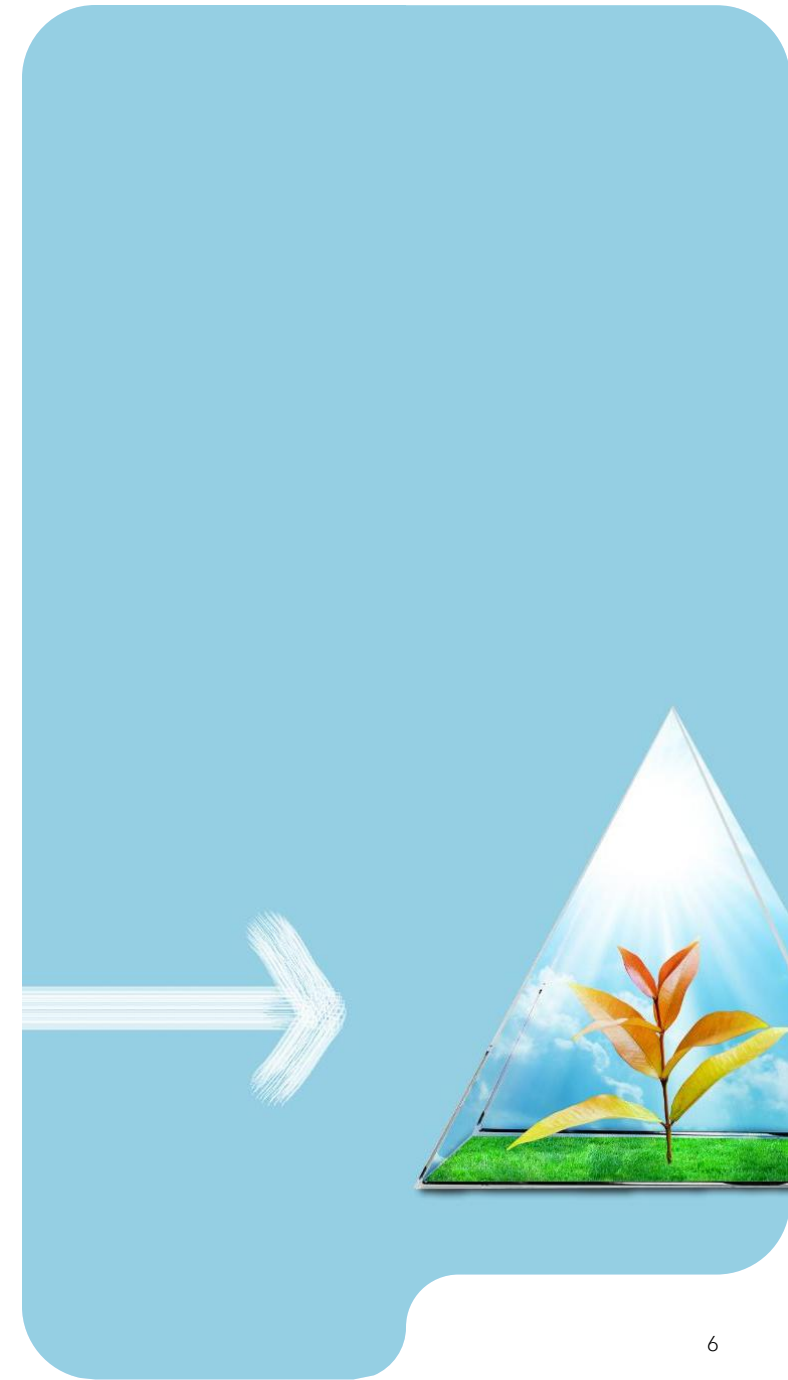
Migration preserves the lifecycle and context of defects beyond simple record import.

Historical Comments Migration

Historical defect discussions and comments are migrated to preserve decision rationale.

Linking Defects to Code

Relationships between defects and code changes are maintained via commit references.



Automation Strategy for Migration

Configuration-Driven Migration

Migration scripts use configuration files to define repository, user mappings, and status translations for automation.

Dry-Run Testing

Dry-run mode generates migration reports without changes, allowing early detection of potential issues and gaps.

Execution and Error Handling

Batch migrations are executed with logging, retries, and error handling to ensure reliability and scalability.



Validation Checklist and Quality Assurance



Verification of Repository Data

Teams verify repository data, default branches, and commit totals to ensure migration accuracy.

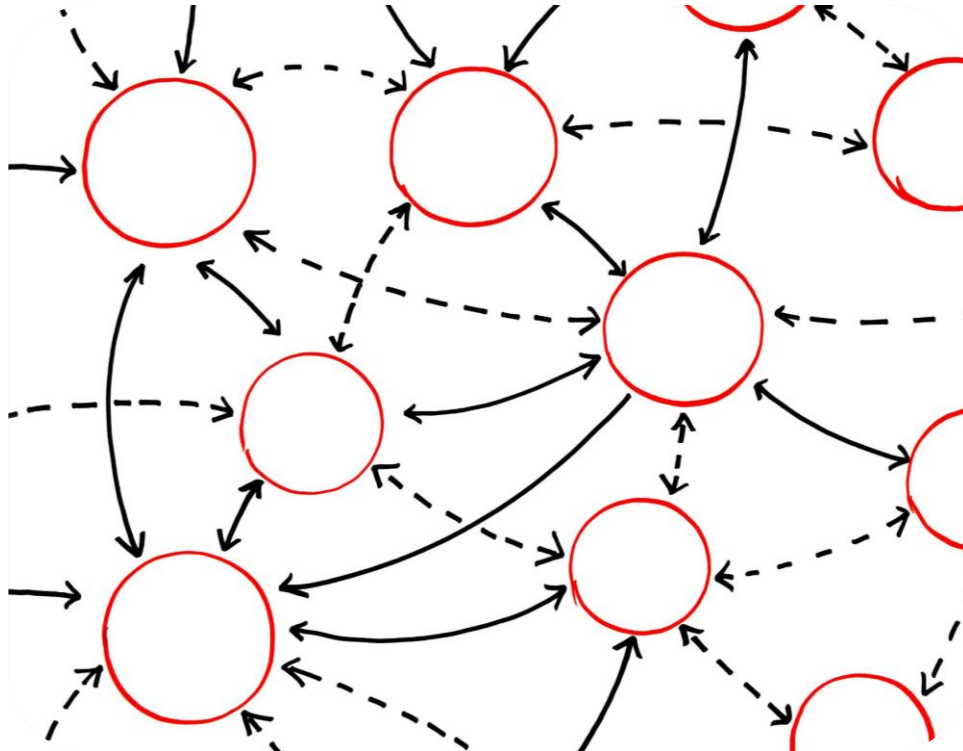
Defect Validation Process

Validation confirms issue counts, states, labels, assignees, and comments match source system data.

Traceability and Exception Reporting

Traceability tested via links from defects to commits; exception reports highlight unmapped users.

Live Demonstration Flow



Migration Pipeline Overview

The demonstration shows the entire migration pipeline from CMVC exports to Git repositories and GitHub issues creation.

Automated Repository Processes

Commits, branches, tags, and comments are generated automatically, showcasing automation in repository management.

Validation and Transparency

Validation reports ensure accuracy while defining expected outputs before execution builds trust in the process.

Practical Application Benefits

The walkthrough helps attendees visualize applying similar tooling in their own environments effectively.

Key Takeaways for Attendees

Migration Planning and Execution

Attendees learn to plan and execute SCM to GitHub migration including source code and defects effectively.

Preserving Code Integrity

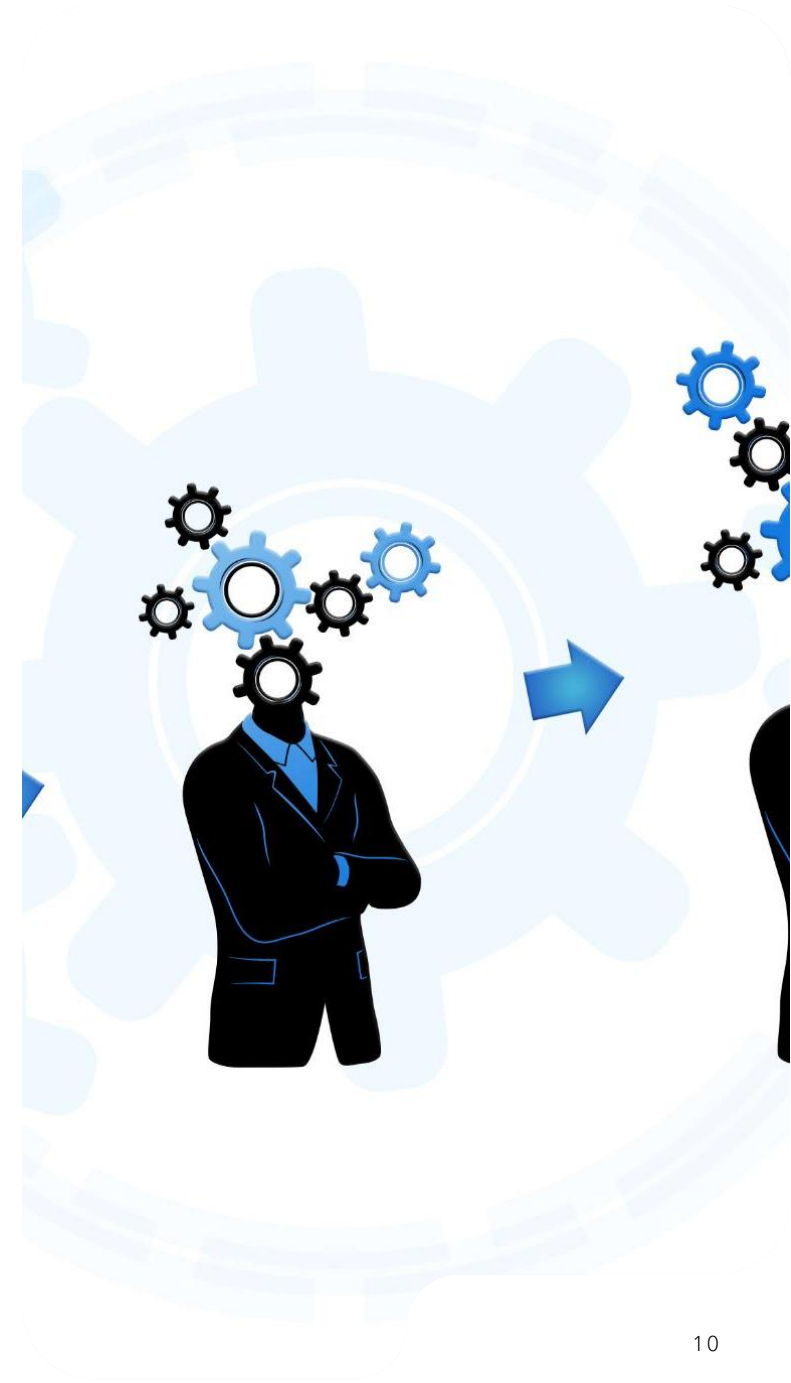
Techniques cover preserving authorship, timestamps, releases, branches, and traceability during migration.

Automation and Validation

Automation enables repeatability and validation ensures accuracy in the migration process.

Transition to Git Collaboration

Insight into moving from centralized SCM to modern Git collaboration models enhancing developer experience.





Conclusion

Modernizing Workflows

Organizations can update development workflows while retaining crucial engineering knowledge and history.

Disciplined Migration

Treat migration as a strategic transformation that integrates collaboration and automation effectively.

THANK YOU